MRG8 - Random Number Generation for the Exascale Era

Yusuke Nagasaka
Tokyo Institute of Technology
Tokyo, Japan
nagasaka.y.aa@m.titech.ac.jp

Akira Nukada
Tokyo Institute of Technology
Tokyo, Japan
nukada@smg.is.titech.ac.jp

Satoshi Matsuoka*
RIKEN Center for Computational Science
Kobe, Japan
matsu@acm.org

Kenichi Miura
Lawrence Berkeley National Laboratory
Berkeley, California, USA
kmiura@lbl.gov

John Shalf
Lawrence Berkeley National Laboratory
Berkeley, California, USA
jshalf@lbl.gov

ABSTRACT

Pseudo random number generators (PRNGs) are crucial for numerous applications in HPC ranging from molecular dynamics to quantum chemistry, and hydrodynamics. These applications require high throughput and good statistical quality from the PRNGs – especially for parallel computing where long pseudo-random sequences can be exhausted rapidly. Although a handful PRNGs have been adapted to parallel computing, they do not fully exploit the features of wide-SIMD many-core processors and GPU accelerators in modern supercomputers. Multiple Recursive Generators (MRGs) are a family of random number generators based on higher order recursion, which provide statistically high-quality random number sequences with extremely long-recurrence lengths, and deterministic jump-ahead for effective parallelism. We reformulate the MRG8 (8th-order recursive implementation) for Intel's KNL and NVIDIA's P100 GPU – named MRG8-AVX512 and MRG8-GPU respectively. Our optimized implementation generates the same random number sequence as the original well-characterized MRG8. We evaluated MRG8-AVX512 and MRG8-GPU together with vender tuned random number generators for Intel KNL and GPU. MRG8-AVX512 achieves a substantial 69% improvement compared to Intel's MKL, and MRG8-GPU shows a maximum 3.36x speedup compared to NVIDIA's cuRAND library.

KEYWORDS

random number generator, exascale, mrg8, parallel

ACM Reference Format:
Yusuke Nagasaka, Akira Nukada, Satoshi Matsuoka, Kenichi Miura, and John Shalf. 2018. MRG8 - Random Number Generation for the Exascale Era. In *PASC '18: Platform for Advanced Scientific Computing Conference, July 2–4, 2018, Basel, Switzerland.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3218176.3218230

*Also with Tokyo Institute of Technology, Department of Mathematical and Computing Sciences.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PASC '18, July 2–4, 2018, Basel, Switzerland
© 2018 Association for Computing Machinery.
ACM ISBN 978-1-4503-5891-0/18/07…$15.00
https://doi.org/10.1145/3218176.3218230

1 INTRODUCTION

In this work, we redesign the MRG8 multiple recursive pseudo random number generator (PRNG) for contemporary wide-vector many-core (Intel Knights Landing) and massively multithreaded GPU accelerators (NVIDIA Tesla), to provide high performance and high quality parallel random number generation for the next generation of HPC systems. A pseudo random number generator (PRNG) is a crucial component of numerous algorithms and applications, including quantum chemistry [3, 5], molecular dynamics, Lattice Boltzman hydrodynamics, and broader classes of Monte Carlo (MC) algorithms. Some key features required by these applications that are identified in numerous publications including Deng and Bowman's 2017 review [2] are

(1) **Long Recurrence Length:** All pseudo random sequences eventually repeat themselves, but you don't want such repetitions to happen during your simulation.
(2) **Good Statistical Quality:** It's important that a random number sequence does not show any biases that would in turn affect the simulation results.
(3) **Deterministic Jump-Ahead for Parallelism:** Any overlap of the random number sequence in parallel threads in a massively parallel system undercut algorithmic efficiency. Any algorithm targeting exascale will need to be ready for exponential scaling of explicit parallelism.
(4) **Performance:** This is desirable, of course, for any computational kernel.

There are a handful of PRNGs that can deliver on the first three of these features, but performance remains elusive [11]. Many-core processors and GPU accelerators are becoming mainstream in a broad array of computing fields. Intel's Xeon Phi series with over 60 cores, and Graphic Processing Unit (GPU) provide tremendous parallelism and performance to users. However, at this time, few random number generators are able to take full advantage of key features of these emerging architectures, such as wide-SIMD vectors in Xeon Phi, or the SIMT execution of the GPU accelerators.

In this work, we reformulate the high quality and well characterized MRG8 PRNG so that it can take full advantage of the features of modern microprocessors and GPUs. We then evaluate

it on the latest computer architectures to compare it with the best available vendor-tuned PRNG libraries on those respective architectures. –Intel Knights Landing (KNL) and the NVIDIA P100 GPU. We introduce the MRG8 PRNG and a reformulated implementations tuned for modern platforms called MRG8-AVX512 and MRG8-GPU, respectively. The random number generation in MRG8 consists of operations between an 8-by-8 matrix and vector, which is tuned to many-core processors. We utilize 512-bit wide registers on KNL. For the GPU, we revise the algorithm for the matrix multiplication step in the original MRG8 to reduce expensive 64-bit operations on GPU. In addition, we focus on reducing the overhead of jump-ahead. This enables to scale the performance and show high throughput on KNL, which executes over a few hundred threads, and GPU, which executes hundreds of thousands of threads.

High performance PRNGs for many-core processors typically utilize two types of APIs – single generation and array generation. The single-generation APIs, such as the standard library of C or C++, generate one random number at a time when the subroutine is invoked. Although single generation suffers from function call overhead and is unable to hide pipeline latencies or make use of SIMD units, the facility is widely used by application developers. In comparison, array generation methods generate multiple random numbers at each invocation, which overcomes subroutine call overhead and also makes the call more amenable to SIMD or GPU accelerators. The array generation method is adopted by Intel MKL and NVIDIA cuRAND, and provides significant random number generator throughput by exploiting SIMD, SIMT, and the high bandwidth memory.

We evaluate the performance of various PRNGs including our own optimized MRG8-AVX512 and MRG8-GPU on two supercomputers, NERSC's Cori with KNL and TiTech's TSUBAME3.0 with GPU acceleration. The performance improvement of MRG8-AVX512 is up to 69% better than other generators of MKL. MRG8-GPU achieves substantially better performance compared to NVIDIA's hand-tuned cuRAND on the NVIDIA P100 platform across a wide-range of sequence sizes for random numbers. MRG8-GPU achieves a maximum of 3.67x speedup compared to cuRAND. Small sequences that fit within on-chip memory are typical for HPC simulation requirements since the random numbers are typically consumed as they are generated, and therefore rarely need to spill out to slower off-chip memory.

The paper is organized as follows. Section 2 covers key metrics used to evaluate the quality of PRNGs, to contextualize the available approaches. Section 3 covers various PRNG algorithms from widely used standards to the state of the art algorithms for many-core processors. Section 4 introduces the mathematical definition of MRG8 and its algorithmic implementation. Section 5 demonstrates how to optimize MRG8 algorithm for many-core processors – especially for Intel KNL and NVIDIA GPU. Finally, Section 6 compares our optimized implementation MRG8 to other state of the art vendor-tuned PRNG libraries.

2 BACKGROUND

The *quality* of random number generation is as essential as peak performance for effective code performance as we march towards exascale computing systems. As noted by L'Ecuyer, et al [11], a generator that delivers high floating-point performance, but fails on any of the below metrics of long-recurrence length, statistical quality, and parallelism will fail to deliver usable performance on future computing platforms. We briefly outline these key metrics, why they are challenging, and how they relate to MRG8.

2.1 Recurrence Length

All PRNGs will eventually repeat themselves. Simple PRNGs such as LCG in the C standard library repeat themselves in as few as 2.15 x 10^9 steps [6]. Therefore a minimum requirement is a PRNG that will not repeat the sequence (or any sub-sequence) for an entire year of executing at full speed on an exascale computing system. If a sequence repeats itself at any point during application execution on any processor in the system, it requires many additional cycles to erase the effect of this auto-correlation, which greatly reduces the effective performance of the algorithm. PRNGs with large recurrence lengths are therefore essential to ensuring *effective* performance of the algorithm.

There are numerous options available to create much larger recurrence lengths that each have their own set of tradeoffs, which we explore in detail in Section 3. One class of PRNGs that provide exceptionally long recurrence lengths and high statistical quality in their sequences are multiple recursive generators (MRGs), which are based on higher-order recursion. MRG8 is an 8th-order recursive MRG that is well characterized and has statistical properties and long-recurrence lengths that are among the best in the field of PRNGs [7].

2.2 Statistical Quality

The sequence must show no statistical bias that would affect the outcome of a simulation. The sequence must offer uniform distribution within the interval of (0,1), which can then in turn be projected into different ranges or other distributions to represent various stochastic processes. In order to investigate the statistical quality of generated random numbers, we summarize the result of TestU01 [12], which is a benchmark set for empirical statistical testing of random number generators. MRG8 passes all of the tests of TestU01 whereas other popular random number generators such as the Mersenne Twister [14] fail. This indicates that MRG8 offers superior statistical quality, which is crucial for code performance. The reformulated MRG8-AVX512 and MRG8-GPU generate exactly the same random number sequence as the original unmodified MRG8 algorithm, therefore prior demonstrations that established the statistical quality of MRG8 are fully relevant to our version.

2.3 Parallelism

Since the demise of Dennard scaling has ceased improvements in processor clock frequencies, the primary way of continuing computing performance scaling has been through explicit parallelism. However, few current PRNGs are well equipped to operate efficiently in massively parallel environments.

There are two primary approaches to parallelizing a PRNG, as outlined in the Philox paper [19]. The first approach is called "multistream" where each generator is given a different random "seed" to produce different random number sequences. Although the overhead of setting the start point is not expensive, there remains a

chance of correlated number sequences that increases substantially with parallelism. The mathematics of the classic "birthday paradox" indicate that this undesirable situation may happen even if the PRNG has an exceptionally long period. The second approach is called "substream," where the PRNG algorithm can deterministically jump ahead in the random number sequence such that each worker on a parallel system can get a sub-sequence that is guaranteed to be non-overlapping with its peers. Since the random number sequences of the workers are guaranteed to be non-overlapping by construction, parallelization does not break the statistical quality of PRNGs on systems that could potentially contain billions of concurrent random number generators. But the jump-ahead can have a sizable overhead, which can hurt parallel scalability if frequent jumps are required.

MRG8 is one of the very few "substream" PRNG algorithms available, which makes it hugely beneficial to have deterministic jump-ahead to ensure there is no overlap of subsequences in a system that could potentially contain billions of concurrent random number generators. However, there is a cost to performing a jump-ahead in MRG8 affects performance. In later portions of the paper, we will dive into the details of how this cost affects the performance of MRG8 in comparison to its peers, and the redesign of the algorithm to mitigate some aspects of that performance challenge.

2.4 Performance

There are three primary issues for scaling the performance of PRNGs on many-core processors. The first issue is that the conventional "single-generation" approach to PRNGs, typified by the commonly used C/C++ standard library and many emerging random number generators (including the original MRG8), is not well suited to efficient use of pipelined processors or high bandwidth memories. A single-generation API is completely oblivious to the complex memory hierarchies and high bandwidth memories seen in recent many-core processors. The second issue is inability of the single-generation interfaces to exploit wide SIMD, which is crucial for performance on modern processors. For example, the Xeon Phi series has 512-bit wide registers, which enable to execute 8 concurrent 64-bit floating point operations per cycle. The third issue is reformulating the algorithm to use the unique features of GPUs such as SIMT (large gangs of parallel threads within each GPU core) and large register files.

Lastly, to exploit the potential computational power of many-core processors, we must take account into the hierarchy of parallelization ranging from the many cores within the chip (both in the case of CPUs and GPUs) and the complexities of the parallelism within each of those cores. The hierarchical nature of the parallelism requires careful design of the jump-ahead mechanism for partitioning the random number sequence among all of the computing elements so as to ensure no overlap in the subsequences, but also bounding the overhead of such operations.

3 RELATED WORK

Linear congruential generator (LCG) is used as rand function in C. LCG is defined as $x_{n+1} = (a \cdot x_n + c)\ mod\ M$, where a, c and M are constant. The next random number is computed from the previous generated random number recursively. The total number of arithmetic operations for computing the next number in the sequence is low, and LCG requires little additional memory space. However, the period of LCG is 2^{32} when $M = 2^{32}$, and the statistical quality is undesirable for certain Monte Carlo applications [6].

Multiple recursive generator (MRG) has a long period and good Lattice structure [4, 8, 16]. The MRG is defined as $x_n = (a_1 x_{n-1} + a_2 x_{n_2} + \cdots + a_k x_{n-k})$ mod p, where a_i is coefficient and p is a prime modulus. The k-th order MRG has the maximal period of $p^k - 1$ and $k-$dimensional equidisribution property, provided that the corresponding characteristic polynomial is primitive. To achieve better statistical properties while maintaining the simplicity of implementation, L'Ecuyer proposed a combined MRGs (CMRGs) method [9]. After extensive searches for good parameter sets [10], L'Ecuyer proposed a CMRG named MRG32k3a that improves performance because it requires only 4 multiplications at each step. MRG32k3a also has a long period near 2^{192} at maximum, and passed numerous empirical statistical tests.

Mersenne Twister (MT) is one of the most popular random number generators [15] because of its exceptionally long period. The algorithm of MT is defined as $x_{n+p} = x_{n+q} + x_{n+1}A + x_nB$, where A and B are matrices. When the period is $2^{19937} - 1$, the generator is called as MT19937. In addition, MT shows good equidistribution in higher dimensions. Thus, we can obtain well-random numbers. The calculation of MT is realized with bitshift, bitwise xor, bitwise or and bitwise and. This largely reduces computation cost, and MT shows good throughput.

There are some derivatives of the original Mersenne Twister. Matsumoto et al. proposed a scheme named Dynamic Creation (DC) for large scale simulations in parallel systems [14]. DC receives unique identifiers (IDs) of the processor, process and machine. These IDs are encoded in the characteristic polynomial, which is a subset of Mersenne Twister. DC is widely implemented and used as MT2203 in vender tuned libraries. Saito et al. proposed a SIMD-oriented Mersenne Twister called SFMT [17]. SFMT is analogous to MT19937, but it tries to exploit some features of modern CPUs such as wide vector register and multi-stage pipelining. SFMT was originally designed for 128-bit registers, and achieved good performance on such systems. Another refinement of the algorithm is the Mersenne Twister for Graphic Processors (MTGP) [18], which supports large state sizes that allow a long period and high-dimensional equidistribution properties, and is able to utilize the high parallelism of GPUs. The authors also proposed a parameter-set generator for MTGP, named MTGP Dynamic Creator (MTGPDC). MTGPDC finds good parameter-sets for MTGP with user specified generator IDs, which are embedded in the recursion parameters of each generator. The generators can yield independent streams, which is useful since each GPU requires many distinct and non-correlated random number streams.

A xorshift RNG approach is one that borrows from cryptography algorithms, and produces a random number sequence with simple operations – exclusive-or and shift [13]. Since the xorshift operations are simple and efficiently executed on computers, the xorshift can generate random numbers at high speed. As an example of exploiting xorshift, the authors proposed a procedure that provides a $2^{128} - 1$ period of random 32-bit integers with only three xorshift operations. Additionally, the statistical quality of the approach is

reasonable. The procedure in C is named XORWOW and adapted to cuRAND.

Philox is a state of the art "counter-based" PRNG for parallel computation [19] that reduces pipeline-latency stalls by eliminating the sequential dependency between new random number generation and generated numbers. The sequence of counter-based numbers is defined as $x_n = b(n)$. Any number of sequence can be generated directly by function b directly from any n. In addition, differing from other existing work, the counter-based approach reduces memory usage since it does not need to store state spaces. The random number sequence of Philox is computed with the operations of mulhi, mullo, xor and addition. The mulhi and mullo output high and low parts of multiplication result respectively. The evaluation result on NVIDIA GPU shows good throughput, and the Philox passes the TestU01 [12] test set – indicating good statistical quality of random number sequence. The Philox is implemented and provided in Intel MKL and NVIDIA cuRAND.

4 MRG8

4.1 Mathematics of MRG8

MRG8 is one of multiple recursive generators [7], which is defined as (1).

$$x_{n+1} = a_0 x_n + a_1 x_{n-1} + \cdots + a_7 x_{n-7} \bmod 2^{31} - 1 \quad (1)$$

In order to have a good Lattice structure, each a_i is chosen as $a_1 = 1089656042$, $a_2 = 1906537547$, $a_3 = 1764115693$, $a_4 = 1304127872$, $a_5 = 189748160$, $a_6 = 1984088114$, $a_7 = 626062218$ and $a_8 = 1927846343$. Although the modulo operation in (1) causes a considerable bottleneck in general, it can be computed with simpler bit-and, bit-shift and add operations when the modulus is a power of 2 minus 1. In addition, (1) can be transformed to a matrix-vector operation, which is represented as (2). $\boldsymbol{y_n}$ is a state vector, which is used for the generation of the next random number. The initial state vector $\boldsymbol{y_0}$ is defined by a seed provided by the user. If we have $A^8 \bmod 2^{31} - 1$, eight random numbers can be computed using one matrix-vector operation as shown in (3).

$$A = \begin{pmatrix} a_0 & a_1 & a_2 & a_3 & a_4 & a_5 & a_6 & a_7 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

$$\boldsymbol{y_n} = \begin{pmatrix} x_n & x_{n-1} & x_{n-2} & x_{n-3} & x_{n-4} & x_{n-5} & x_{n-6} & x_{n-7} \end{pmatrix}$$

$$\boldsymbol{y_{n+1}} = A\boldsymbol{y_n} \bmod 2^{31} - 1 \quad (2)$$

$$\boldsymbol{y_{n+8}} = A^8\boldsymbol{y_n} \bmod 2^{31} - 1 \quad (3)$$

Figure 1 shows the compute-efficient implementation of the MRG8 algorithm. Since a_i and x_i are represented by 31-bit integers, multiplication between a_i and x_i takes 62 bits. Summation of 8 multiplications requires 65 bits, which 64-bit machine cannot handle efficiently. Therefore, the summation is divided into two parts, s1 and s2 in Figure 1. Bit-shift and bit-and operations are executed for each and then summed up, in order to perform Modulo $2^{31} - 1$ computations (Line 9-11).

MRG8($\boldsymbol{a}, \boldsymbol{y_n}$)

```
1  // Generate a 64-bit floating point random number ∈ [0, 1)
2  // a and y_n are vectors with size of 8
3  MASK ← (2^31 − 1)
4  s1 ← 0
5  s2 ← 0
6  for q ← 0 to 3
7    do s1 ← s1 + (a_q * x_{n−q})
8       s2 ← s2 + (a_{q+4} * x_{n−(q+4)})
9  s ← (s1&MASK) + (s1 >> 31) + (s2&MASK) + (s2 >> 31)
10 s ← ((s&MASK) + (s >> 31))
11 s ← ((s&MASK) + (s >> 31))
12 r ← double(s − 1)/MASK
13 x_{n+1} ← s
```

Figure 1: Efficient Algorithm of MRG8

4.2 Jump-ahead Sequence

On parallel random number generation, randomly setting the start point of sequence for each worker has a risk of overlapping random number sequences, which incapacitates to secure statistical quality of random numbers. Thus, the whole sequence used among all workers should be divided in order not to overlap, and then each worker jumps ahead to the start point for each. MRG8 provides jump-ahead function to the i-th start point as (4).

$$\boldsymbol{y_i} = A^i\boldsymbol{y_0} \bmod 2^{31} - 1 \quad (4)$$

Since the expensive jump-ahead function can reduce the advantages of parallelization, the computational cost of the jump-ahead function needs to be minimized. In MRG8, A^{2^n} $(0 \le n \le 246)$ are precomputed and stored to a file. At the initialization phase of MRG8, the precomputed matrices are read from the file as necessary. Figure 2 shows the algorithm for the jump-ahead the sequence. To reduce the computational complexity, the jump-ahead function is computed using matrix-vector operations rather than matrix-matrix operations.

JUMP-AHEAD($A, \boldsymbol{y}, i$)

```
1  // y is initial state vector (= y_0)
2  // i is jump distance represented in up to 246-bit
3  for j ← 0 to 246
4    do if (i&(0x1)) == 1
5         then y ← A^{2^j} y mod 2^31 − 1
6       i ← (i >> 1)
```

Figure 2: Jump-ahead Algorithm of MRG8

5 OPTIMIZING MRG8 FOR MANY-CORE PROCESSORS AND GPUS

Multicore and Many-core processors with wide SIMD and GPU-accelerators have become the primary compute engine of many recent supercomputers. To accelerate random number generation on these platforms, we have reformulated the MRG8 algorithm for the Intel KNL and NVIDIA GPU to utilize the architecture specific facilities and enormous parallelism on these emerging architectures. We will start by showing the techniques utilized for the Intel Knights Landing (KNL) and next for the NVIDIA PASCAL (P100) GPU.

Finally, we describe the API for this optimized implementation of MRG8.

5.1 Vectorizing MRG8 for KNL

KNL is the 3rd generation of the Xeon Phi family. KNL has over 60 cores and supports 4 threads for hyper-threading. The Xeon Phi series implements the AVX-512 instruction set with 512-bit wide registers. KNL has 96GB of DDR4 memory that operates at nearly 90GB/s to provide memory capacity, and also 16GB of high bandwidth MCDRAM that offers a peak bandwidth of over 400GB/s. The MCDRAM can be configured for three different memory modes; Cache, Flat and Hybrid. In Cache mode, MCDRAM acts as a direct-mapped cache of DDR4 main memory. DDR4 and MCDRAM are regarded as a NUMA hierarchy when in Flat mode, but Flat mode is rarely used in practice. In Hybrid mode, some amount of MCDRAM is in a separate address space from the DDR4 like Flat mode, and the remaining part of the MCDRAM is used as cache for the DDR4. To achieve high throughput on KNL, it is crucial to exploit both the high parallelism and the bandwidth MCDRAM. When MRG8 is parallelized on KNL, each thread executes the jump-ahead function and computes the initial state vector for each start point. As described in the prior section, our algorithmic approach enables the jump-ahead function of MRG8 to be computed with low overhead. This better enables the tuned MRG8 to exploit high parallelism of the Intel KNL.

To achieve further performance improvement, vectorization with AVX-512 is indispensable for the KNL architecture. Figure 3 shows our approach to vectorizing MRG8. Our target operation for vectorization is $\boldsymbol{y_{n+8}} = A^8 \boldsymbol{y_n} \bmod 2^{31}-1$, and matrix-vector operation is executed as an outer product. Each vector and matrix with 8 elements are treated as 64-bit, and they fit into a 512-bit wide register. To keep state vector $\boldsymbol{x}$ on the register and reduce data movement between memory and register, the matrix A^8 is transformed to A_{KNL} and the vector $\boldsymbol{x}$ is permuted on the register after multiplication and addition. Since all of vector operations in Figure 3 are executed as AVX-512 instructions, vectorized MRG8 takes full advantage of the wide vector registers on KNL.

5.2 Optimizing MRG8 for the NVIDIA GPU

GPU computation is organized around gangs of many threads that execute the same code in a Single-Instruction Multiple-Threads (SIMT) manner. The unit of SIMT computation is specified as a bundle of 32 threads, which is called a "warp." Any divergence in the instruction stream for the threads in the warp greatly reduces execution performance, so our optimized MRG8 algorithm for the GPU simultaneously generates 32 random numbers at a time in each invocation (5) to match the size of the warp.

$$A_{GPU} = \begin{pmatrix} A^8 \\ A^{16} \\ A^{24} \\ A^{32} \end{pmatrix}, \quad \begin{pmatrix} \boldsymbol{y_{n+8}} \\ \boldsymbol{y_{n+16}} \\ \boldsymbol{y_{n+24}} \\ \boldsymbol{y_{n+32}} \end{pmatrix} = A_{GPU}\ \boldsymbol{y_n} \bmod 2^{31}-1 \qquad (5)$$

Figure 4 shows the MRG8 algorithm for the GPU. Matrix-vector operation is computed as outer product. Since the memory access to an element of the matrix must be coalesced into consecutive accesses to achieve good performance, the matrix A_{GPU} is stored in column-major order. When executing multiplication between

$$A_{KNL} = \begin{pmatrix} \boldsymbol{a_1} & \boldsymbol{a_2} & \boldsymbol{a_3} & \boldsymbol{a_4} & \boldsymbol{a_5} & \boldsymbol{a_6} & \boldsymbol{a_7} & \boldsymbol{a_8} \end{pmatrix}$$
$$= \begin{pmatrix} a_{00} & a_{01} & a_{02} & a_{03} & a_{04} & a_{05} & a_{06} & a_{07} \\ a_{11} & a_{12} & a_{13} & a_{14} & a_{15} & a_{16} & a_{17} & a_{10} \\ a_{22} & a_{23} & a_{24} & a_{25} & a_{26} & a_{27} & a_{20} & a_{21} \\ a_{33} & a_{34} & a_{35} & a_{36} & a_{37} & a_{30} & a_{31} & a_{32} \\ a_{44} & a_{45} & a_{46} & a_{47} & a_{40} & a_{41} & a_{42} & a_{43} \\ a_{55} & a_{56} & a_{57} & a_{50} & a_{51} & a_{52} & a_{53} & a_{54} \\ a_{66} & a_{67} & a_{60} & a_{61} & a_{62} & a_{63} & a_{64} & a_{65} \\ a_{77} & a_{70} & a_{71} & a_{72} & a_{73} & a_{74} & a_{75} & a_{76} \end{pmatrix}$$

MRG8-AVX512($A_{KNL}, \boldsymbol{y_n}$)

```
1  // Generate eight 64-bit floating point random numbers
2  MASK ← (2^31 − 1)
3  s1 ← 0
4  s2 ← 0
5  for q ← 0 to 3
6    do s1 ← s1 + a_q y_n
7       // PERMUTE(x) returns w s.t. w[i] = x[(i + 1)%8]
8       x ← PERMUTE(y_n)
9  for q ← 4 to 7
10   do s2 ← s2 + a_q y_n
11      x ← PERMUTE(y_n)
12 s ← (s1&MASK) + (s1 >> 31) + (s2&MASK) + (s2 >> 31)
13 s ← (s&MASK) + (s >> 31)
14 s ← (s&MASK) + (s >> 31)
15 y_{n+8} ← s
16 r ← (double)(s − 1)/MASK
```

Figure 3: Vectorized MRG8 Algorithm. Row/Column ids in matrix A_{KNL} are corresponding to original A^8. The variables in bold type represent vectors with 8 elements, which are executed as a single AVX-512 instruction

32-bit integers and producing a 64-bit integer, the GPU emulates the 64-bit multiplications using many 32-bit operations, whereas Intel KNL supports such operations at the hardware level. To reduce expensive mixed-precision integer multiplications we use the __umulhi() instruction, which computes multiplication between 32-bit unsigned integers and produces upper 32-bit of result, and outputs lower 32-bit integer in a separate step. Along with this optimization, bit shift and mask operations are also separated into the upper and lower 32-bit, which are efficiently executed as 32-bit operations. This optimization largely reduces computation cost from the multiplications between 32-bit integers outputting 64-bit integer.

In MRG8 for the GPU architectures, the matrix-vector operation for the jump-ahead sequence is executed as inner product that is executed by the 32 threads of the warp. Figure 5 demonstrates how to jump ahead to each start point on GPU. Each warp cooperatively executes the jump-ahead function where the 8-by-8 matrix-vector operation is divided into upper and lower parts, and computed by all 32-threads of the warp. Each thread computes one multiplication, and multiplied results are summed up by using warp shuffle instructions, which is for sharing data across the same warp. This optimization for the GPU largely reduces the computational cost of the jump-ahead operation.

MRG8-GPU($A_{GPU}, \boldsymbol{y_n}$)

```
1  // A_GPU is (A^8 A^16 A^24 A^32)^T
2  // Generate i-th random number, r_i
3  MASK ← (2^31 − 1)
4  t ← i%32
5  l ← 0
6  u_1 ← 0
7  u_2 ← 0
8  for q ← 0 to 3
9    do l ← l + a_{t,q} · x_{n−q}
10       l ← l + a_{t,q+4} · x_{n−q−4}
11       u_1 ← u_1 + __umulhi(a_{t,q}, x_{n−q})
12       u_2 ← u_2 + __umulhi(a_{t,q+4}, x_{n−q−4})
13 s ← ((u_1 + u_2) << 1) + (l&MASK) + (l >> 31)
14 s ← ((s&MASK) + (s >> 31))
15 s ← ((s&MASK) + (s >> 31))
16 x_{n+1+t} ← s
17 r_i ← double(s − 1)/MASK
```

Figure 4: MRG8 Algorithm for GPU. Variable with underline is 32-bit integer.

Jump-Ahead-GPU($A, \boldsymbol{y}, i$)

```
1  // y is initial state vector (= y_0)
2  // i is jump distance represented in up to 246-bit
3  // a^j_{p,q} represents (p, q) element of A^j
4  tid ← threadId%32
5  rid ← tid/8
6  cid ← tid%8
7  MASK ← (2^31 − 1)
8  for j ← 0 to 246
9    do if (i&(0x1)) == 1
10           then for k ← 0 to 1
11                  do s_k ← a^{2^j}_{rid+4∗k,cid} · x_cid
12                     s_k ← s_k + __shfl_xor(s_k, 4)
13                     s_k ← s_k + __shfl_xor(s_k, 2)
14                     s_k ← (s_k&MASK) + (s_k >> 31)
15                     s_k ← s_k + __shfl_xor(s_k, 1)
16                     s_k ← (s_k&MASK) + (s_k >> 31)
17                     s_k ← (s_k&MASK) + (s_k >> 31)
18                if cid == 0
19                  then x_rid ← s_0
20                       x_{rid+4} ← s_1
21                i ← (i >> 1)
```

Figure 5: Jump-Ahead Algorithm for MRG8-CUDA

On GPUs, the number of threads and thread block size must be appropriately chosen to maximize performance, but the warp requires a jump-ahead function to ensure the random number subsequences do not overlap, which becomes a source of overhead for MRG8. Therefore, we have manually selected the optimal number of threads needed to take full advantage of GPU parallelism while simultaneously reducing the overhead of the jump-ahead function. In the future, we will explore auto-tuning to automatically find the optimal threadblock size. Another option we intend to explore in future work is a hierarchical approach to the jump-ahead to improve scalability by bounding the overhead of jump-ahead to Log(nwarps).

void rand_parallel(**double** *ran, **int** n)

```
1  // state is the initial state vector (=y_0)
2  tnum ←omp_get_max_threads();
3  for tid ← 0 to tnum in parallel
4    do n_tid ← n/tnum
5       start ← n_tid ∗ tid
6       state_tid ← state
7       jump_ahead(start, state_tid)
8       rand(ran + start, state_tid)
```

Figure 6: Example of parallel array generation in MRG8

5.3 MRG8 API

MRG8 is implemented in C++, and the variables and functions are managed within a C++ class. The code provides two types of rand functions respectively for sequential (without state vector), and for parallel (with state vector) situations. It also includes one function for jump-ahead as shown below.

- **double** rand();
- **double** rand(**unsigned int** state[8]);
- **void** rand(**double** *ran, **int** n);
- **void** rand(**double** *ran, **int** n, **unsigned int** state[8]);
- **void** jump_ahead(**int** jump_val, **unsigned int** state[8]);

The first function is a *single generation* API where each function call returns a single random number. This follows the format of the C and C++ standard API for random number generators. However, unlike the standard libraries of C and C++, our MRG8 implementation supports random number generation with multi-threading – even for the sequential interface. The user determines the total number of random numbers to generate per invocation, and then each thread jumps ahead to its respective start point by using the jump_ahead function. After that, each thread can use its own state vector to generate random numbers in parallel.

The second interface is an *array-generation* API where the user provides a pointer to the array along with the array size, and then the generator fills that array with random numbers according to the size argument. This API for array generation is similar to that adopted by Intel MKL and cuRAND. Since generating multiple random numbers per invocation (*array-generation*) reduces the overhead of the function call, random numbers can be generated much more efficiently using this interface. Figure 6 shows example pseudo code for the multithreaded *array-generation* algorithm for MRG8 using OpenMP. Similar to the single-generation interface, each thread computes the corresponding state vector for the start point by using the jump_ahead function. After that, each thread can use its own state vector to generate random numbers in parallel.

Both MRG8-AVX512 and MRG8-GPU are implemented as subclasses of the MRG8 base class so that the APIs of both MRG8-AVX512 and MRG8-GPU are same as original MRG8. Thus, an application developer can easily apply our optimized MRG8 library to their applications since the API of MRG8 is almost the same as the C++ standard library, which is also typically used by vender tuned libraries such as NVIDIA cuRAND and Intel MKL.

Table 1: Random number generators provided by the standard libraries

	Mersenne Twister	MRG32k3a	XORWOW	Philox
C++11	MT19937	-	-	-
Intel MKL	MT19937, MT2203, SFMT	✓	-	✓
NVIDIA cuRAND	MT19937, MTGP32	✓	✓	✓

Table 2: Overview of Evaluation Environment

	Cori (KNL)		**TSUBAME3.0**
CPU	**Intel Xeon Phi Processor 7250**	**GPU**	**NVIDIA Tesla P100-SXM2 GPU**
#Sockets	1	#SM	56
#Cores/socket	68	#CUDA Cores	3584
Clock	1.4GHz	Clock	1.3GHz (Boost: 1.4GHz)
L1 cache	32KB/core	Shared memory	64KB/core
L2 cache	1MB/tile	L2 cache	4MB
Memory			
	96GB DDR4 and 16GB MCDRAM		16GB HBM2
Software			
OS	SuSE Linux Enterprise Server		SUSE Linux Enterprise Server 12 SP2
Compiler	Intel C++ Compiler (icpc) ver18.0.0		CUDA ver.8.0.61
Option	-g -O3 -qopenmp		-g -O3 -arch=sm_60

6 EVALUATION

We evaluate our optimized MRG8 library and compare it to four other state of the art methods from the viewpoint of the speed of random number generation and statistical quality of random numbers on the Intel KNL and NVIDIA P100 platforms.

6.1 Experimental Setup

6.1.1 Target Random Number Generators. We use the C++11 standard library and the Intel MKL library for the evaluation on Intel KNL, and the NVIDIA cuRAND on the GPU systems. Table 1 summarizes which generator is supported by each library. MKL and cuRAND provide optimized generators for SIMD or GPU for the original Mersenne Twister, MT19937. Both MRG32k3a and Philox are also supported in MKL and cuRAND. All evaluations in our paper generate 64-bit floating point random numbers with a uniform distribution. All performance numbers presented represent the average of ten executions in order to account for system variability.

6.1.2 Evaluated Systems. Our evaluated systems are summarized in Table 2. We used two supercomputers for the evaluation; one is Cori at NERSC and the other is TSUBAME3.0 at GSIC. Cori consists of both Haswell nodes and KNL nodes, but we only use KNL nodes for this evaluation. All tests use the KNL's MCDRAM in *Cache mode*. The TSUBAME 3.0 has 4 NVIDIA P100 GPU in each node. We use one KNL socket for the MRG8-KNL evaluation and one GPU socket for the MRG8-GPU evaluation.

6.2 Model for Performance Upper Bound

To provide more insight into the measured performance of our MRG8 implementation, we consider the performance upper bound of the array type random number generation. The performance bound is determined as $\min(p_m, p_c)$; p_m for a memory-bound use case and p_c for a compute-bound use case.

Firstly we consider the memory-bound case. In this case, the performance is restricted by memory bandwidth for storing the generated random numbers to memory. The evaluation is done on a write-only 1Gigabyte stream of 64-bit floating point data using the STREAM benchmark, and the derived upper bound of the random number generations per second (RNG/sec). The array that stores the random number sequence is allocated with function call _mm_malloc() in order to ensure optimal alignment for the KNL's cache, and with cudaMalloc() on the GPU. In our evaluation, KNL achieves 166.6GB/s when operating in cache mode, and the P100 GPU achieves 570.5GB/s. These high bandwidth memories enable random numbers to be generated throughputs up to 22.4 billion RNG/sec on KNL and 76.6 billion RNG/sec on P100 GPU for any of the random number generators, provided they are not compute-bound.

Next is regarding the compute-bound case. We only model MRG8-AVX512 and MRG8-GPU since we don't have access to the detailed implementation of the other random number libraries (they are not open-source). To simplify the problem, we only consider the most time-consuming part of the kernel, and excludes the jump-ahead overhead. In MRG8-AVX512, 44 instructions are executed in the core kernel of MRG8. The Intel Xeon Phi has 136 vector units (2 units/core), and the performance upper bound is 1.4[GHz] $*136 * 8/44 = 34.6$ billion [RNG/sec] if each instruction can be executed in a single cycle. When the array size can fit entirely into the Level 1 data cache, MRG8-AVX can execute about 54% more [RNG/sec] than the high-speed memory-bound case. In MRG8-GPU, 101 instructions are executed. Our target NVIDIA P100 GPU architecture has 3584 CUDA cores, and with a raw performance

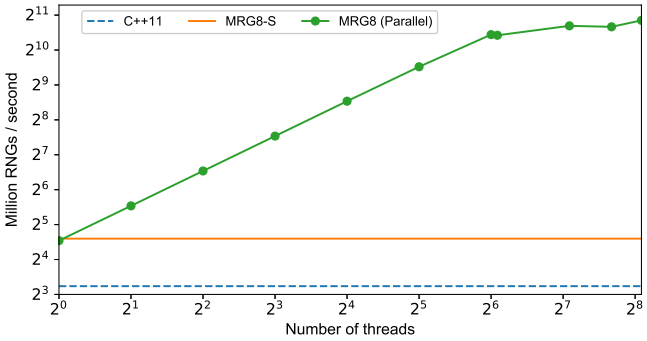


Figure 7: Performance of random number generators on KNL. One random number is generated by one function call.

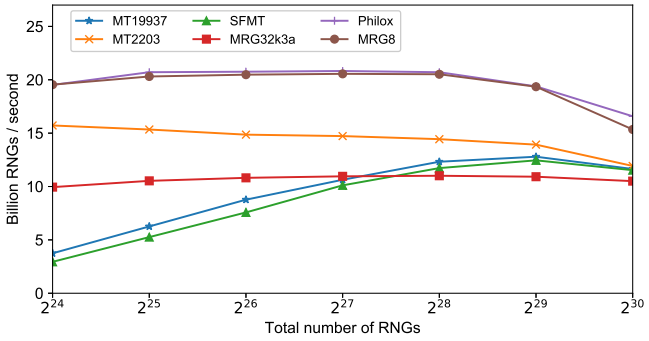


Figure 8: Performance of random number generators on KNL for large number sequences

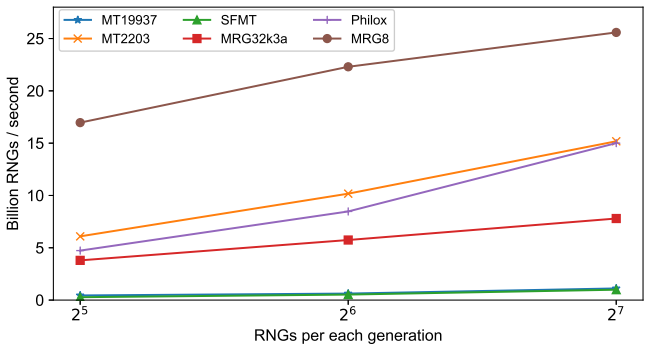


Figure 9: Performance of random number generators on KNL with multi-thread for small random number sequences

upper bound of $3584 * 1.4/101 = 49.7$ billion [RNG/sec]. From this analysis, MRG8-GPU is a compute-bound kernel in all cases.

Lastly, the jump-ahead can add additional overhead to the execution, but we do not include it in the model because in a practical application we can always set the jump-ahead period such that it constitutes a negligible impact on code performance.

6.3 Performance on the KNL Architecture

6.3.1 Single Generation. Figure 7 shows the performance results for generating 2^{24} random numbers. The C++11 results use mt19937, and MRG8-S refers to the sequential implementation of MRG8. MRG8 (Parallel) is the multi-thread version using the single generation API. Even though KNL has many cores and executes many threads, C++11 and MRG8-S with the single thread are still relatively inefficient on the platform. On the other hand, MRG8 (Parallel) can scale the performance well to 68 threads, and achieve superior throughput in comparison to C++11. Since the performance of the single generation API is restricted by the overheads of function calls and the latency of the compute units themselves, hyper-threading does not provide further performance improvements beyond 68 threads.

6.3.2 Array Generation for Large Sequences. Figure 8 shows the performance for generating random number sequences from 2^{24} to 2^{30} in length. The codes run with 68, 136, 204, and 272 threads, and we select best performance for each generator and size of random number sequence. Although MT19937 shows superior performance for a single thread, its performance advantage drops for larger thread counts – especially for sequences larger than 2^{24} elements. MRG8 shows comparable performance to Philox, which are both close to the upper bound for memory bandwidth. The stream benchmark result for 2^{30} elements of 64-bit floating point (= 8GB) elements is only 149 GB/sec. Philox and MRG8 performance degrades when generating more than 2^{30} random numbers per invocation due to the stream bandwidth limit.

6.3.3 Array Generation for Small Sequences. As we see Figure 8, the performance of array generation for large sequences of random numbers is limited by memory bandwidth. Next, we consider repeated generation of short random number sequences that is more typical of application codes. Figure 9 shows the result of array generation for short sequences on 272 threads. Each thread generates 2^5, 2^6 or 2^7 random numbers 1000 times. Since the memory requirement for the array is only 1KB per thread, each algorithm should be primarily compute-bound. In this regime, MRG8 is 1.69x faster than the other random number generations at 2^7 array lengths.

6.3.4 Scalability of Array Generation. Figure 10 shows the scalability results for generating 2^{24} random numbers. Test trials are performed with 1, 2, 4, 8, 16, 32, 64, 68, 136, 204 and 272 threads. In both situations, MT19937 and SFMT performance goes down after 64 threads. When evaluated for strong scaling, the number of random numbers generated by each thread goes down as the thread count goes up and the consequent cost of jump-ahead becomes more dominant. While MT19937 and SFMT require a large computing cost for jump-ahead, other generators such as Philox and our MRG8 see little overhead for the jump-ahead operation. In particular, MRG8 shows good scalability beyond 68 threads.

6.3.5 Cost of Jump-ahead in Array Generation. Figure 11 shows the ratios of time spent in jump-ahead compared to the total execution time for random number generations. The evaluation is done on 272 threads. The cost of jump-ahead is relatively small when the total size of the random number sequence is large. Otherwise, jump-ahead becomes a performance bottleneck. As we can

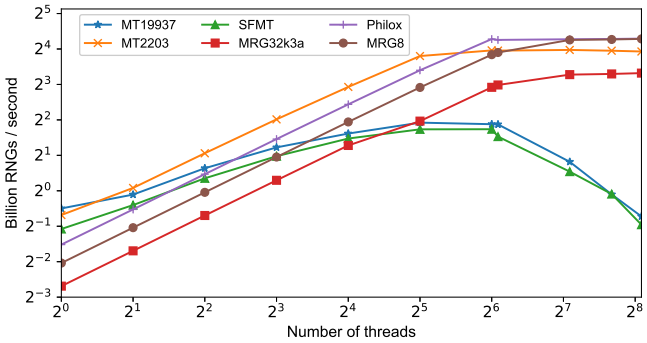


Figure 10: Scalability result of random number generators on KNL (2^{24} random numbers)

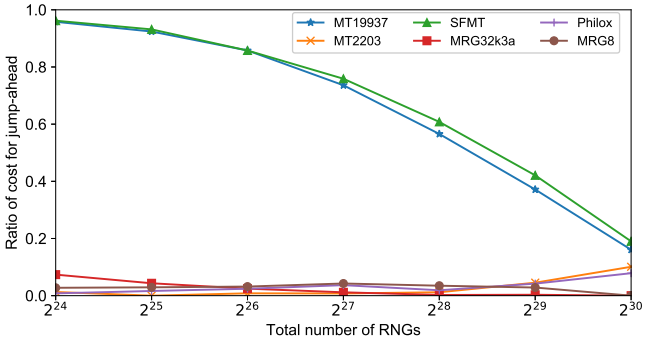


Figure 11: Overhead of jump-ahead on KNL

see, MT2203, MRG32k3a, Philox and MRG8 require little cost for jump-ahead for most sequence lengths. On the other hand, the jump-ahead sequence becomes serious bottleneck on MT19937 and SFMT, which limits the parallel scalability of those random number generators.

6.4 Performance on the GPU Architecture

6.4.1 Throughput of Array Generation. Figure 12 shows the evaluation result on the NVIDIA P100 GPU. We selected the parameter settings for thread count and thread block size that achieve the best results for MRG8-GPU; total number of threads from $\{2^i | i = 10, 11, \ldots, 20\}$ and thread block size from $\{2^i | i = 5, 6, \ldots, 10\}$. MRG8-GPU achieves high throughput for nearly any random number sequence length. On the other hand, other generators such as Philox show high throughput only for large sequence lengths. Since many real-world applications have other data structures that occupy memory, it is important to achieve high throughput also for small sequence lengths. MRG8-GPU works for 2^{24} sequence lengths, which requires 128MB. MRG8-GPU achieves 3.36x speedup compared to other generators in cuRAND for 2^{24} random number lengths. Even for 2^{30} sequence lengths, MRG8-GPU shows 5% performance improvement.

6.4.2 Parameter Tuning of MRG8-GPU. Parameter tuning of the number of threads for MRG8-GPU is one of the key factors for performance. We vary the total number of threads and thread block

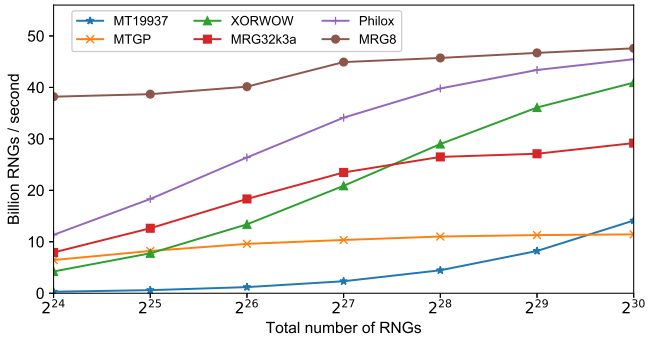


Figure 12: Performance of the random number generators on the P100 GPU

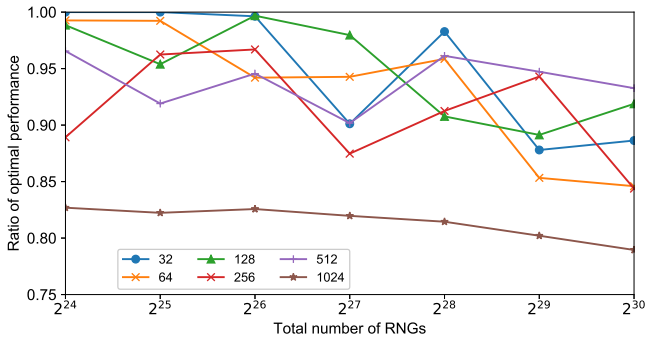


Figure 13: Performance difference between best and heuristic parameter setting (2^{17} threads)

size as parameters, and pick the best setting in Figure 12. However, when we consider a more realistic use case for a typical library, only one parameter setting must be selected for the duration of execution. Therefore, we must devise a proper heuristic to select the total number of threads, and compare the performance of the heuristic to the best setting found by the exhaustive search. On the P100, it has 56 SMs where each SM executes 2048 threads. Thus, it is necessary to execute over 114,688 threads to achieve optimal performance. From this insight, we evaluate MRG8 with 2^{17} and 2^{18} threads. Figure 13 and 14 show the relative performance with each thread block size and compare it to the best parameter setting found through search. For short random number sequences from 2^{24} to 2^{26} elements, the total number of threads of 2^{17} works well, and the setting of 2^{18} threads is close to the optimal value for larger sequence lengths. We can achieve sufficiently good performance with a thread block size of 32 if we select an appropriate total number of threads. From this result, it is feasible for us to set a thread block size of 32 and set the total number of threads to 2^{17} or 2^{18} to enable near-optimal performance of MRG8 for a wide range of sequence sizes. Autotuning can only offer a modest increase in delivered performance over this simple heuristic.

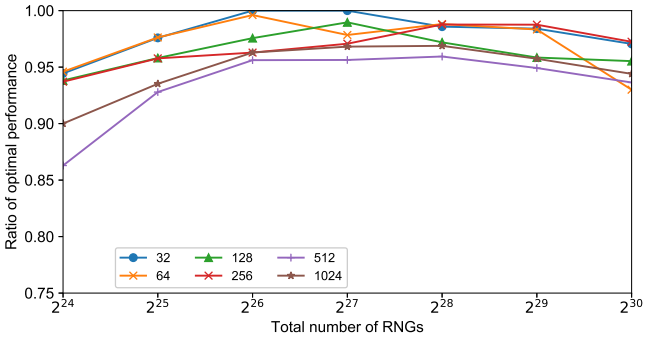


Figure 14: Performance difference between best and heuristic parameter setting (2^{18} threads)

6.5 Memory Usage of MRG8

The memory bandwidth of many-core processors is high, but the capacity is limited. Fortunately many applications consume the random numbers as they are generated, and therefore do not require that the PRNG itself occupies much memory capacity, but there are exceptions. Since requirements vary, we examine the full range of cases for how much memory is used in MRG8-AVX512 and MRG8-GPU.

Firstly we discuss about MRG8-AVX512. There are two types of requirements. One is shared data among all threads, and the other is thread-private data. The matrices A^8 and A^{2^i} $(0 \le i \le 246)$ are constant data, and accessed by all threads. Since A^8 is stored as 8-byte to fit into 512-bit register and A^{2^i} as 4-byte in MRG8-AVX512, the memory requirement for these matrices is 62.25KB. The state vector is thread-private data, and it is only 64 bytes. In summary, MRG8-AVX512 requires 235 bytes per thread on 272 threads.

Next is regarding MRG8-CUDA, where the situation is almost same as MRG8-AVX512. The matrices, A_{GPU} and A^{2^i} $(0 \le i \le 246)$ are allocated on global memory of the GPU. All of the matrices in MRG8-CUDA are stored as 4-byte, and the memory usage is 62.75KB. In MRG8-CUDA, each warp (= a bundle of 32 threads) requires a state vector of eight 32-bit elements from the definition, but to reduce warp divergence, which degrades the performance, we allocate 32 * 4 bytes memory space for each warp. In short, MRG8-GPU requires no more than 5 bytes per thread on 2^{17} threads.

6.6 Quality of Random Numbers

Random number generator is required not only generation speed but also having long period and good statistical quality. Table 3 summarizes the quality of each random number generator. This data of the analysis comes from their original or investigation paper, and instruction of vender libraries [1, 10, 13–15, 17–19]. The period is based on the mathematical definition of each generator, and they all have long periods. Especially, a family of Mersenne Twister has an extremely long period, but the period of MRG8 is also sufficient. For the evaluation of statistical quality of random number generator, TestU01 is widely used. As briefly mentioned in Section 2.2, TestU01 is the most comprehensive test suite to date for empirical testing of the uniform random numbers. It was developed by P.L'Ecuyer and R.Simard at the University of Montreal and is implemented in ANSI C [12]. It consists of three pre-defined battery of tests as follows:

(1) Small Crush: 15 tests, using 2^7 random numbers
(2) Crush: 186 tests, using 2^{35} random numbers
(3) Big Crush: 234 tests, using 2^{38} random numbers

All test cases are passed by only MRG32k3a, Philox and MRG8. In this paper, we reimplement MRG8 for KNL and GPU, but they generate same sequence of random numbers as the sequential version. Therefore, the statistical quality of our MRG8 for many-core processors is secured.

Table 3: Quality of each random number generator

	Period		Statistical quality
	MKL	cuRAND	
MT19937	$2^{19937}-1$	$2^{19937}-1$	
MT2203	$2^{2203}-1$	$2^{2203}-1$	
SFMT19937	$2^{19937}-1$	-	
MTGP	-	2^{11214}	
XORWOW	-	$(2^{160}-1)2^{32}$	
MRG32k3a	2^{191}	$> 2^{190}$	✓
Philox	2^{130}	2^{128}	✓
MRG8	$(2^{31}-1)^8-1$	$(2^{31}-1)^8-1$	✓

7 CONCLUSION

MRG8 is a high quality PRNG that offers key qualities of statistical uniformity, efficient parallelism, and long recurrence length. The reformulated algorithm provides substantial performance benefits (69% faster than Intel's vendor tuned MKL and 3.36x faster than NVIDIA's vendor-tuned cuBLAS) for a broad array of exascale-class applications, and overcomes the challenges of avoiding cross-correlations among the random number sequences for systems that may contain hundreds of millions to billions of independent PRNG streams. In our follow-up work, we will work on reducing the overhead of performing jump-aheads in the RNG stream. Since the statistical quality of the random number sequence offers no guarantee of good application behavior, our follow-up work will also include application demonstrations to demonstrate the value of this approach.

ACKNOWLEDGEMENT

This work was partially supported by JST CREST Grant Number JPMJCR1303 and JPMJCR1687, and performed under the collaboration with DENSO IT Laboratory, inc., and performed under the auspices of Real-World Big-Data Computation Open Innovation Laboratory, Japan. The Lawrence Berkeley National Laboratory portion of this research is supported by the DoE Office of Advanced Scientific Computing Research under contract DE-AC02-05CH11231. One of the authors (KM) would like to thank Prof. Pierre L'Ecuyer of Montreal University for providing the 8th order primitive polynomial for this study.

CODE

Our implementations of the MRG8 algorithm on GPU and KNL are available at https://github.com/kenmiura/mrg8.

REFERENCES

[1] L Yu Barash and Lev N Shchur. 2014. PRAND: GPU accelerated parallel random number generation library: Using most reliable algorithms and applying parallelism of modern GPUs and CPUs. *Computer Physics Communications* 185, 4 (2014), 1343–1353.

[2] Lih-Yuan Deng and Dale Bowman. 2017. Developments in pseudo-random number generators. *Wiley Interdisciplinary Reviews: Computational Statistics* 9, 5 (2017), e1404–n/a. https://doi.org/10.1002/wics.1404 e1404.

[3] WMC Foulkes, L Mitas, RJ Needs, and G Rajagopal. 2001. Quantum Monte Carlo simulations of solids. *Reviews of Modern Physics* 73, 1 (2001), 33.

[4] James E Gentle. 2006. *Random Number Generation and Monte Carlo Methods*. Springer Science & Business Media.

[5] Brian L Hammond, William A Lester Jr, and Peter James Reynolds. 1994. *Monte Carlo Methods In Ab Initio Quantum Chemistry*. World Scientific.

[6] Peter Hellekalek. 1998. Good random number generators are (not so) easy to find. *Mathematics and Computers in Simulation* 46, 5 (1998), 485–505.

[7] Kenta Hongo, Ryo Maezono, and Kenichi Miura. 2010. Random number generators tested on quantum Monte Carlo simulations. *Journal of computational chemistry* 31, 11 (2010), 2186–2194.

[8] Donald E Knuth. 1997. The art of computer programming, 3rd edn. seminumerical algorithms, vol. 2. (1997).

[9] Pierre L'Ecuyer. 1996. Combined Multiple Recursive Random Number Generators. *Operations Research* 44, 5 (1996), 816–822.

[10] Pierre L'ecuyer. 1999. Good Parameters and Implementations for Combined Multiple Recursive Random Number Generators. *Operations Research* 47, 1 (1999), 159–164.

[11] Pierre L'Ecuyer, David Munger, Boris Oreshkin, and Richard Simard. 2016. Random numbers for parallel computers: Requirements and methods, with emphasis on GPUs. (06 2016).

[12] Pierre L'Ecuyer and Richard Simard. 2007. TestU01: A C library for empirical testing of random number generators. *ACM Transactions on Mathematical Software (TOMS)* 33, 4 (2007), 22.

[13] George Marsaglia et al. 2003. Xorshift RNGs. *Journal of Statistical Software* 8, 14 (2003), 1–6.

[14] Makoto Matsumoto and Takuji Nishimura. 1998. Dynamic Creation of Pseudorandom Number Generators. *Monte Carlo and Quasi-Monte Carlo Methods* 2000 (1998), 56–69.

[15] Makoto Matsumoto and Takuji Nishimura. 1998. Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation (TOMACS)* 8, 1 (1998), 3–30.

[16] Harald Niederreiter. 1992. *Random Number Generation and Quasi-Monte Carlo Methods*. SIAM.

[17] Mutsuo Saito and Makoto Matsumoto. 2008. SIMD-Oriented Fast Mersenne Twister: a 128-bit Pseudorandom Number Generator. In *Monte Carlo and Quasi-Monte Carlo Methods 2006*. Springer, 607–622.

[18] Mutsuo Saito and Makoto Matsumoto. 2013. Variants of Mersenne Twister Suitable for Graphic Processors. *ACM Transactions on Mathematical Software (TOMS)* 39, 2 (2013), 12.

[19] John K Salmon, Mark A Moraes, Ron O Dror, and David E Shaw. 2011. Parallel Random Numbers: As Easy as 1, 2, 3. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC), 2011*. IEEE, 1–12.